TakeThemOut: A stealth and dialogue driven game

Edoardo Vassallo
edo.vas@protonmail.com

April 15, 2026

1 Details of the proposal

1.1 Full specification of your proposal

The objective of the project is the development of a simple game using the Godot Game Engine, in coordination with the VEsNA toolkit and intent-based chatbots.

The game is composed of two main sections, one stealth based and one dialogue based.

In the stealth section, the human controlled player must navigate a top-down maze-like level patrolled by a set of agent-controlled NPCs. The final objective of the player is to reach a specified point of the level without being caught by one of the agents. The level will contain multiple ways for the player to avoid from the agents, such as closets to hide into.

There are currently three types of agents planned to be implemented:

- Sentinels (∼8 instances): Stays still in a point, changing its point of view. When the player is noticed, it will try to call the attention of another agent nearby. If no agent is nearby, the sentinel will stop trying to contact after a short while.
- Patrols (∼4 instances): The agent moves around the maze in a pre-determined pattern. When the player is noticed, it will try to either follow the player and catch them or move towards a nearby supervisor and inform them. When it is alerted of the player's sighting, it will move towards its location.
- Supervisors (∼2 instances): The agent moves slowly around the maze, in a longer pattern. When the player is noticed, or another agent contacts them, the supervisor will remotely alert all of the patrols of the player's sighting.

When an agent follows the player, it will try to reach it until it stops making eye-contact with it. Afterwards, the agent will stop looking for the player after a short while. After being alerted, an agent will act more aggressively for a short while, moving faster, acting more random in his movement options, and checking more often hiding spots for the player.

When a player is caught, the game is over, and they have to start from the beginning of the stage. Before starting though, the player can have a small natural language (English) interaction with the agent which caught them, through a chatbot. Based on the player's response, the agent will react either positively or negatively. Its behaviour will change next round, being either more forgiving or more aggressive. Each individual agent will have a slightly different personality, subtly communicated through its design. The player will have to look for the best response for each agent.

In the second section, the player will have to interact in natural language (English) with an intent-based chatbot. The player will be tasked with a series of questions to get answers to. The chatbot will implement emotional states, and be initially not collaborative. The player will have to put him at ease in order to get him to get the information needed. The number of sentences which could be recognized are not yet been clearly defined, they are currently thought to be in the order of 20-40

1.2 The kind of your proposal

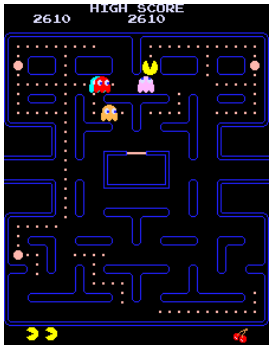
The proposal is a joint SDAI-NLP creative project.

1.3 The range of points/difficulty of your proposal

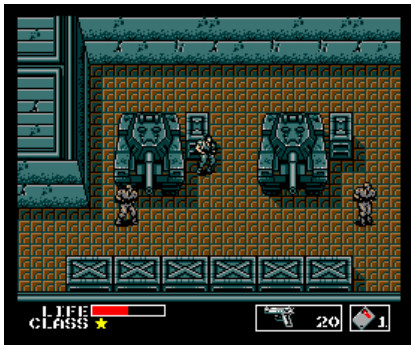
The difficulty of the proposal was not directly stated at the time of assignment. It is considered by the student hard.

2 Introduction

2.1 Project Motivation



(a) A screenshot from *Pac-Man* (1980)[1]



(b) A screenshot from *Metal Gear* (1987) [2]

Stealth games are a genre of videogames which find their origins in classics of the medium, such as *Pac-Man* (1980) and officially begin with *Metal Gear* (1987). In these games, the player is usually tasked with traversing a level populated with various enemies. The player is usually unable to attack the enemies, or rewarded for refraining from it. As such, they must navigate the environment avoiding contact with the enemies.

Usually the types of enemies encountered in such games are various, in order to offer a more interesting experience, but individual instances of the agents are identical between each other. As such, while this helps the player in learning their patterns and mastering the game mechanics, it may result in a reduced sense of immersion, with the player treating the NPCs as predictable bots or cameras, as they are in a sense.

In this project, multiple technologies are used to improve the perceived personality and character of an example stealth game. While the game itself is implemented through the Godot Engine, the VEsNA-ProTemper tool-kit is used in order to write the agents' minds in Jason's AgentSpeak, abstracting their reasoning and enabling the design of more complex behaviours, while also exploiting the temper feature to easily characterize the agents' behaviour based on their set personality and mood. The latter is influenced by the player's actions, giving a subtle feedback, and opening the possibility of more nuanced strategies exploiting the agents' personality.

Figure 2: A screenshot from *Façade* (2005)[3].

Moreover, inspired by the ambitious game *Façade* (2005), the player has also the possibility of interactiong in natural language with a set of the agents, through the usage of Rasa chatbots. While the attached game mechanics could be implemented with simpler and less cumbersome methods, the ability of writing directly to the bot increases the perceived intelligence of the

system by the player, and with it, their immersion.

2.2 Report Structure Overview

Given the heterogenous structure of the codebase, the following report will be divided in multiple sections. First, the abstract structure of the overall game, its objectives, the game loop and such, will be explained. Then, each part of the system will be focused on, the Godot implementation, the AgentSpeak scripts and the Rasa chatbots. Lastly, eventual integration details will be discussed, with the testing and evaluation of the whole system.

3 Project Overview

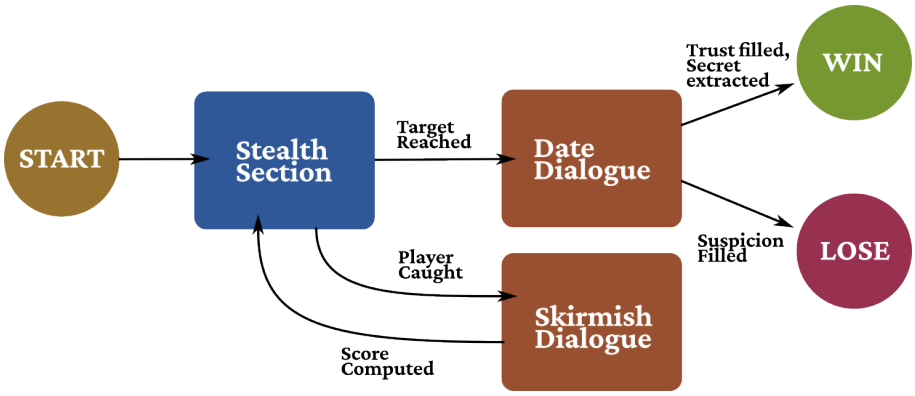


Figure 3: Schema detailing the different phases of the game, and the transition between them

The final game can be divided in three sections:

- **Stealth Section**: This section is a 2D isometric stealth game, in which the player must navigate a maze avoiding the enemy agents' point of view, and reach a target character. If the player is caught by some of the agents, a skirmish dialogue section will start. If the player reaches the target character, the date dialogue section will start.
- **Skirmish Dialogue**: In this section, the player has the opportunity to talk with the agent which caught them. The player can send a single sentence in natural language. Based on the sentence, and the agent's personality, they will receive an adequate answer, and a score will be computed for the interaction. After that, the stealth section will start again from the beginning, with the computed score used to alter the relative agent behaviour. In the final project, the player can dialogue only with the Patrol agents.
- **Date Dialogue**: In this section, the player must speak with the target character, an agent. Two meters are shown to the player, a "Trust" meter and a "Suspicion" meter. The objective of the player is, over the course of multiple natural language exchanges, fill the former and keep the latter empty. At each sentence, the agent will answer accordingly, hinting to the correct strategy to adopt. The player should ask the agent for a specified secret. Given that the Trust meter is filled, and the suspicion is low enough, the agent will answer with the secret, and the game will be considered won. If before that, the suspicion meter fills up completely, the game will be considered lost, and the player will have to start again from the Stealth Section

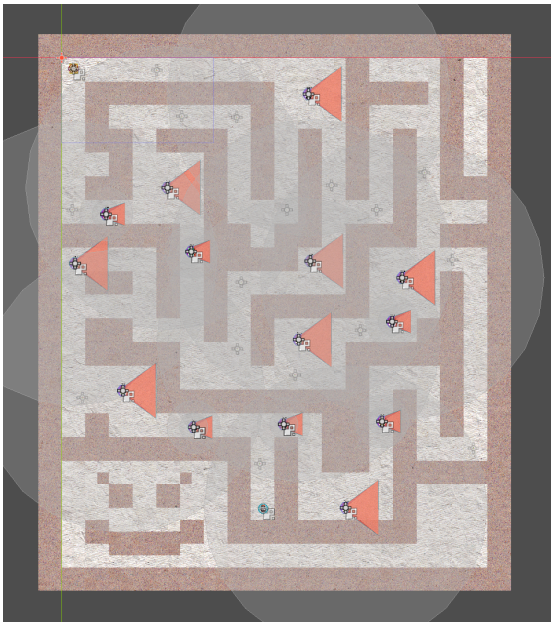


Figure 4: Screenshot of the maze where the the stealth section is set.

4 Stages and Bodies (Godot)

The main game was built in Godot Engine, version 4.6. The topic of this section are the systems built directly in the engine, the stages, the player, and most importantly, the agents' bodies.

4.1 Stages

Two stages were created for the game, the 2D isometric stage for the stealth section, and the dialogue scene, used for both the date and skirmish scenes.

- The **stealth scene** (Figure 4) is a simple maze, built using Godot's tileset system. Each of the tiles which make up the floor contain a navigation region tile, which allow agents to move through the scene using Godot's navigation systems. Each of the tiles which make up the walls contain instead collision tiles.
- The **dialogue scene** (Figure 5) mainly contains a textbox, with a section for user input. The name and sprite of the interlocutor is displayed. Two buttons are used to send input and return to the previous scenes. During the date scene, the trust and suspicion meters are displayed on the top left of the screen.

4.2 The Player

The player is built on top of a simple CharacterBody2D node. It can simply move in the four cardinal directions. Its most prominent feature is the **crumb mechanic** (Figure 6). Every time the player has moved a set amount of space, a "crumb" will be spawned on its location, a simple collision node with position and timestamp, which will disappear after a set amount of time. This crumbs will be later used by the agents for tracking the player.

4.3 Agent Bodies

The agents were built following the structure of the player, with a CharacterBody2D node at their bases. They also possess a cone of vision, and some other features specific to each enemy

Figure 5: Screenshot of a Skirmish dialogue scene

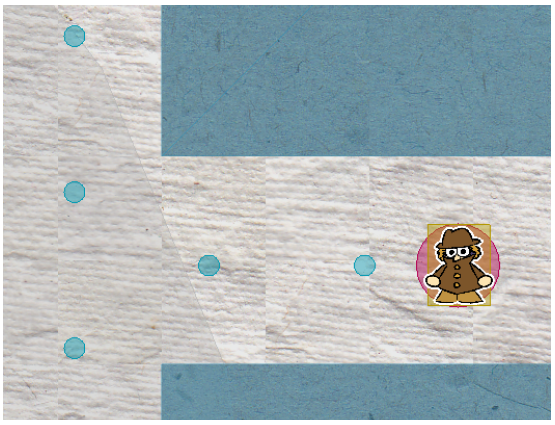


Figure 6: A screenshot of the stealth section, with debug visuals activated. The player and its trail of "crumbs" can be seen

type.

More importantly, their reasoning structure follows the State Machine paradigm, with a set of states, subprocesses, which the agents transition between, based on their mind commands or external events. The usage of states could appear simplistic, reducing the direct impact of the mind, moving part of its duties to the bodies and requiring synchronization between the two. It will be shown that this choice is needed to avoid problems given by latency between the mind and the body, making the agents more reactive and engaging to play against. Moreover, the state machine structure is a classic approach used in the game development space for agents, and as such it reflects a realistic development environment.

Now, we will see the behaviour and structure of each enemy type, while also introducing their abstract role in the game.

4.3.1 The Sentry (Previously Sentinel)

The objective of this agent is to stand still in a spot and alternate between a set amount of points of view. If it spots the player, it should alert its allies nearby.

This behaviour, on the Godot side, is implemented through the following set of states:

- **Idle State**: During this state, the agent does nothing. This is often used as a transition state, while waiting for commands from the mind.
- **Scan State**: During this state, the agent switches between the different point of views, at a set rhythm, which can be passed when entering the state. If the player is detected, the mind is notified, and the body transitions to Idle, waiting for orders.
- **Alert State**: When entering this state, the agent disables its vision cone and performs a scan for allies nearby, using a ShapeCast2D node. The list of allies is passed to the mind, while the agent waits for a set amount of time. This wait time is needed to avoid detection messages spam, and giving the player time to escape. At the end of this wait, the mind is notified of the end of the alert, and the body transitions to Idle, waiting for orders.

This is the simplest of the agents, unable to move and catch the player, dependant on cooperation with its allies. The details on the mind's side of the logic will be discussed later, in the section focusing on AgentSpeak.

4.3.2 The Patrol

The objective of this agent is to patrol the corridors of the maze and catch the player. The agent moves between a set of specified points. If an ally calls them, they will move to the alert point, and then return to their patrol.

If the player is spotted during its patrol, it will try to catch them. If the player gets out of sight, the agent will follow its crumb trail. After following the trail for some time, it will investigate the zone around the last crumb, and ultimately return to patrol. If the player is seen again, it will return to chasing them.

A more complex feature is the "messenger system". If the agent is following the player, and sees an ally which is not alerted, it will ask them to go alert a captain, their superior. The second agent will navigate to the nearest captain and alert them, while the first agent will keep following the player, and if it meets another un-alerted agent, it will not ask them again.

This global behaviour, on the Godot side, is implemented through the following set of states:

- **Idle State**: During this state, the agent does nothing. This is often used as a transition state, while waiting for commands from the mind.
- **Patrol State**: During this state, the agent moves towards a point, using Godot's navigation system. The point towards it moves is specified when entering the state, and it could

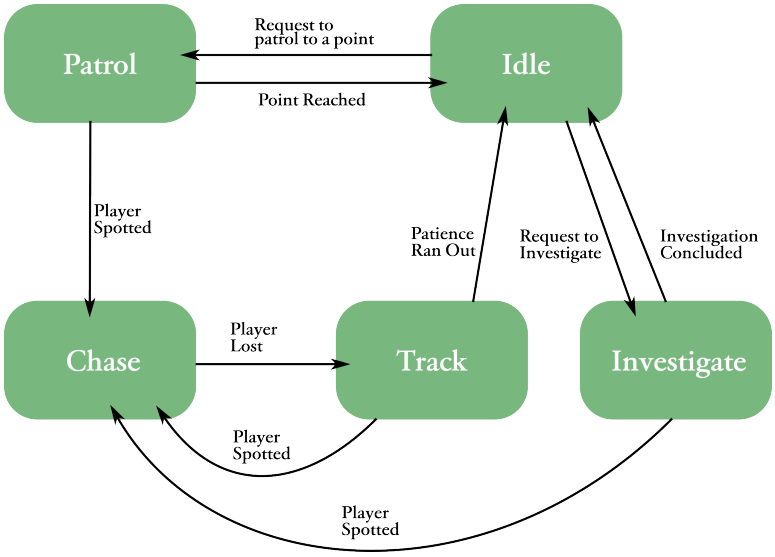


Figure 7: Schema depicting the transitions between the body states of the Patrol agent's state machine

be the next, previous or random point in its set of patrol points, a direct 2D coordinate or another agent's position, the latter used for the messenger system. When the point is reached, the mind is notified, and the body transitions to the Idle state. If during the patrol, the agent sees the player, it notifies the mind, *and moves directly to the Chase state*.

- **Chase State**: During this state, the agent moves directly towards the player's position. When entering this state, a patience value is passed, which is stored and used later. If the player gets out of sight, the agent transitions to the Track states.
- **Track State**: During this state, the agent follows the trail of crumbs left by the player, always prioritizing the newest available. The agent detects the crumbs through a ShapeCast2D. If the player is seen during this, the agent notifies the mind and moves back to the Chase State. The number of crumbs followed is counted, and when it gets bigger than the patience value, the agent notifies the mind, and transitions to the Idle state.
- **Investigate State**: When entering this state, the agent generates a set number of random points around its position. Then, it moves between them. If the player is seen during this, the agent notifies the mind and moves to the Chase State. When the agent has visited all the points, it notifies the mind, and moves to the Idle State.

It can be seen how this agent is way more complex than the previous. One interesting detail is the relative **dissonance** with the mind: When the player is seen, the body enters on its own the Chase state, without waiting for the mind approval. Moreover, when entering the Track state, the mind is not even notified, resulting in the two states being the same from the mind's point of view. This is done in order to compensate for the latency between mind and body in communication. If the body were to wait for the mind's direct orders for each of these transitions, we'd lose precious time which is needed when chasing the player. As such, the idea is to transition directly to the Chase state. If the mind agrees, it will ask to transition to it again, if the mind objects it can send a message to transition out of it. Similarly, we cannot lose time when moving between Chase and Track, so Track is treated as a "ghost state" to the mind, and the patience value it needs is passed in the Chase state.

4.3.3 The Captain (Previously Supervisor)

This agent can be seen as a extended version of the previous, and as such it reuses most of its code. Similarly to the Patrol, The Captain move between the maze, looking for the player. Instead of relying directly on a set of patrol points, it asks the other agents for the last player's sightings. After collecting them, it computes the average point, and move there. If the player has not been sighted, it will move towards a random patrol point. The reason for this behaviour is to make the captains more unpredictable. While the player may quickly learn the behaviour of the patrols, they will still need to contend with the captains

If the Captain sees the player, before moving to the Chase state, and behave similarly to the Patrol, it will notify other agents of the player's sighting, and ask them to come over. This adds even more unpredictability, breaking the Patrols' patterns.

Given its similarities to the Patrol, we will not repeat here its state structure.

5 Agent Design (AgentSpeak)

The minds of the agents we just described were realised in AgentSpeak, through the usage of the VEsNA Pro-Temper Toolkit, needed to communicate with Godot. Four agents were realised, three of which correspond to the three types of enemies of our game, and one more, called "ready agent", used for synchronising at startup with the Godot side, and distributing shared updates.

Before taking a look at each agent on its own, we consider some of the shared features developed for the agents.

5.1 Receiving Updates and Sending Actions

In order to communicate with the bodies, `VesnaAgent.java` was modified, adding multiple handlers for the messages sent by the bodies. We list here the message types which can be received:

- **Sight**: Sent when the player is seen, it can contain its 2D position. It gets translated into a `sight(object, id, pos(x, y))` belief, added to the agent memory
- **Allies Found**: Sent with the list of allies found nearby the agent. It gets translated into a `allies_nearby([ally1, ally2, ...]` belief.
- **Navigation Completed**: Sent when the body has reached a previously requested position, contains the status of the navigation and the reached point. It gets translated into a `navigation(status, point_name)` belief.
- **Target Lost**: Sent when the player is lost during tracking, it contains the last position and the reason for losing them. It gets translated into a `target_lost( pos(X,Y), reason` belief.
- **Setup**: Sent at the beginning of the game to the ready agent, contains a set of mood values to communicate to the other agents. It gets translated into a `sympathy_updates([ [sentry1, 0.5], ... ]` belief.

At the same time, the agents possess a set of internal actions, used to send messages to the body. This has been implemented as helper classes, put in the `vesna/via` folder. The actions are the following:

- **Transition To**: This method is used to transition between the states of the agent's body. It takes as argument the name of the state to transition to, and some optional parameters for the state. An example usage is `vesna.transition_to("Patrol", [target("resume")]);`

- **Set Variable**: This method is used to set a variable in the agent's body to a specific value. It requires the name of the variable and the new value. An example usage is `vesna.set_var(switch_time, 2.0);`
- **Add Temper**: This action is used to add a specific value to a mood. It requires the name of the mood variable, and the value to add. In order to it to work, a new `modifyMood` method was added to the `Temper` class, to directly modify the mood to a specified value.
- **Signal Ready**: This action is used by the Ready agent at startup, to signal to the Godot side that the Jason agents have successfully started.

5.2 Personality and Mood

In order to characterize and diversify the behaviour of our agents, the temper feature of VEsNA was used extensively. In particular, two personality traits were implemented:

- **Aggressiveness**: This personality trait controls how confrontational the agent is, how much and how fast does it intervene to stop the player.
- **Laziness**: This personality trait controls how constant the agent is, for how long does it chase during encounters, for how long does it pauses during patrols, etc.

These two permanent traits are used together with two other mood traits:

- **Fear**: This mood trait is incremented by the encounters with the players, and reduced by successful chases, patrols and interactions with other allies. This mood interacts with the other traits, by making the choices more extreme. When fearful, a more aggressive agent may decide to act alone, disregarding their allies, while a lazy agent may decide to avoid intervening altogether.
- **Sympathy**: This mood trait is modified by the result of the interactions with the player in the skirmish dialogue sections. It controls how lenient is the agent with the player. A more sympathetic agent may let the player go, even if they are an aggressive type, while the opposite can happen for a lazy agent with low sympathy.

So, while the personality traits profile the individual instance of the agent, the addition of the mood traits adds dynamic behaviours. Additionally both of the latter are modified through the interaction with the player, giving them direct feedback for their actions, and handing them another tool to use at their advantage.

The main place in which to look for the temper's effect are the Patrols. In the game, four patrol agent instances have been placed, each with a different high-low Aggressiveness-Laziness personality combination. This is the better example to see how the behaviour is impacted by the temper system.

5.3 Agent Minds

In this section we now examine each of the four agent files individually, focusing on the structure of their decision-making and on how the temper system, introduced in the previous subsection, concretely modulates their behaviour. We will not discuss every single plan and component of each, since it would result tedious, and not very useful. The goal is to identify the key decision points each agent faces and illustrate how personality and mood bend the outcome at those points.

5.3.1 Ready Agent

The `ready_agent` is the simplest of the four, and its role is only infrastructural. On startup, it waits briefly to allow the rest of the Jason runtime to initialise, then fires a `vesna.signal_ready` action to notify the Godot side that the multi-agent system is online.

Its second responsibility is the distribution of sympathy values at the start of each game run. When Godot sends a `sympathy_updates` belief containing a list of agent-value pairs, the ready agent walks the list recursively and dispatches an `update_sympathy` goal to each named agent in turn. This ensures that the personality state carried over from a previous run's chatbot interactions is correctly restored before any NPC begins patrolling. The ready agent has no other roles and takes no part in gameplay after this initialisation phase.

5.3.2 Sentry

The sentry is a purely reactive agent: it has no movement loop and issues no goals of its own. Its entire decision cycle is driven by incoming events, making it the most straightforward mind to follow.

When a `sight(player, Id, pos(X, Y))` belief is added, multiple detection plans can fire depending on the agent's personality profile. An aggressive sentry reacts immediately and experiences a small reduction in fear, reinforcing its confrontational tendency, while a fearful one startles, losing time, and accumulates additional fear. In all plans, the actual response is delegated to the `!alert_about_player` intent.

Here we can see again the impact of temper. Four plans compete for selection, each producing a call to `vesna.transition_to("Alert", ...)` with a different `duration` parameter:

- An **aggressive** sentry remains in alert for 12 seconds and reduces its own fear, confirming the encounter.
- A **fearful** sentry panics and raises a short 5-second alert, consistent with an agent that wants help quickly rather than acting alone.
- A **lazy** sentry issues only a 3-second alert, doing the minimum required.
- A **sympathetic** sentry, notably, produces only a 1-second alert, effectively suppressing the response. Here we can see how high sympathy manifests not as leniency but as active deception.

Another instance occurs in the captain-reporting plans. When the captain broadcasts a `report_sightings` request, a sympathetic sentry replies with `sighting_report(none)` regardless of whether it holds a `last_player_pos` belief, deliberately withholding its intel. The default plan reports the true position. This means that a player who has built up sympathy with a sentry effectively gains a blind spot in the captain's intel.

After an alert concludes, the recovery plans govern the return to scanning. A fearful sentry resumes with a fast scan interval (`switch_time` of 2.0), while a lazy one relaxes to a slow 6.0-second rhythm.

5.3.3 Patrol

The patrol is the most complex agent in the system, combining a continuous movement loop, a chase mechanism, a post-chase investigation routine, and a messenger sub-system for escalating sightings to the captain. Moreover, it uses a set of beliefs to record the currents state of the agent, and act accordingly. We now examine each of its systems in isolation.

Movement loop. On startup, the patrol transitions its body to the `Patrol` state and enters a movement cycle, in which when it reaches a spot, triggers `!patrol` and `!decide_next_step` to decide where to move next. At each waypoint arrival, the agent calls `!rest_at_waypoint` before issuing the next movement. The rest duration is temper-dependent: an aggressive patrol pauses for 0.5 seconds, a lazy one for 6 seconds, and a sympathetic one for 8 seconds. An aggressive patrol may also, with a 30% probability, backtrack to the previous waypoint to check its rear, adding unpredictability for the player.

Chase system. When a `sight` belief arrives and the patrol is not already chasing, it drops its current intentions, records the player's position, and calls `!start_chase`. The key parameter passed to the body is `patience`, which controls how long the body will pursue before giving up:

- A **sympathetic** patrol issues a patience of 2, feigning effort before abandoning the chase.
- An **aggressive**, low-sympathy patrol pursues with a patience of 25, making it extremely persistent.
- A **lazy** patrol settles at 5.
- The default is 10.

When the body loses the target, a `target_lost` belief triggers `!investigate_area`, where the patrol sweeps a number of nearby points before giving up. Again, sympathy and laziness compress the search to a single point, while a vengeful patrol checks up to 8, making it genuinely dangerous even after losing line of sight. Once the investigation concludes, a `signal_investigation` belief resets all chase-related state flags and returns the agent to its patrol loop.

Messenger system. When a patrol is actively chasing and detects nearby allies through the `allies_nearby` belief, it attempts to recruit one of them as a messenger to relay the sighting position to the captain. The recruiting patrol sends a `become_messenger` achieve-goal to the first ally in the list. The receiving agent can accept or reject the role:

- If idle, it accepts, stores the player's position in `messenger_report_pos`, drops its patrol intentions, and navigates toward the nearest captain using a dedicated `find_captain` target.
- If already chasing or already acting as a messenger, it rejects the request, which causes the recruiting patrol to lock its `messenger_failed` flag for the remainder of the chase, avoiding repeated fruitless attempts.

If no captain can be found, or if the captain moves out of range during navigation, the messenger aborts and returns to its own patrol. The captain, for its part, explicitly rejects the `become_messenger` goal, ensuring that the command hierarchy is never asked to run errands. This whole mechanism allows the patrols to communicate sightings to its superiors while lacking direct communication with the captain at all times.

5.3.4 Captain

While the patrol reacts to what it sees, the captain actively seeks information from its allies. Its main loop is an intel-gathering cycle: at each waypoint arrival, instead of simply choosing the next target, the captain broadcasts a `report_sightings` achieve-goal to all agents in the system, waits two seconds for replies to accumulate, and then analyses the collected `sighting_report` beliefs.

The analysis step uses an helper `vesna.calc_centroid` function to compute an average of all positions reported by the squad. The result is the point of highest recent activity on

the map, and the captain navigates toward it. This makes the captain more dangerous to the player: even if it never directly sees the player, it will gradually converge on the area where the most sightings have occurred. An aggressive captain moves to the centroid with full urgency; a fearful one introduces a delay before committing; a lazy captain discards the intel entirely and continues its own route; a sympathetic captain ignores the reports and moves to a random waypoint, sabotaging the coordination.

On direct player contact, the captain's temper determines whether it broadcasts the sighting to the entire squad before starting the chase. An aggressive captain immediately issues a full broadcast and enters a high-patience chase. A sympathetic captain starts only a short private chase with no broadcast, letting the player escape a global response.

6 Chatbots (Rasa)

For the interactions in natural language with the agents, intent-based chatbots written in RASA where chosen.

The reason for the usage of specifically intent based ones, compared to others who make direct use of LLMs for answering the user, is simple: While generative AI offers more variability in its choice of words, making the interactions more natural and fitting to the player's input, this same variability reduces authorial intent. In story-driven games, where such a component would most likely feature, the tone and choice of words of a NPC's dialogue is an essential tool to communicate its personality and character, and as such it must be under absolute control of the writer. Hence, we intend to leverage the intelligence of neural networks only for understanding the relevant intent of the player's interaction and deduct what meaning should the bot's answer communicate, leaving the choice of adequate wording to the game writer's pen.

Two chatbots were written for the game, one for the Skirmish dialogue section, and one for the Date dialogue section. Being developed alongside each other, they share some common structures. Still, we will now focus on each on its own.

6.1 Skirmish Chatbot

The Skirmish chatbot is the simpler of the two, whose logic is shared between all enemy agents during the stealth section. Its functioning is episodic: At each time, it receives a sentence from the player, and based on the sentence, and the personality preset of the current agent, selects an adequate answer and computes a relative score.

6.1.1 Intent Classification Pipeline

The intent classification pipeline structure is shared between both chatbots, and is built on top of SpaCy's `en_core_web_lg` model as its linguistic backbone. The full pipeline is composed of the following components, in order:

- **SpaCy Tokenizer and Featurizer**: Tokenises the input and produces dense feature vectors by mean-pooling their pre-trained word embeddings.
- **LexicalSyntacticFeaturizer**: Complements the dense vectors with sparse features capturing lexical and syntactic patterns at the sentence level.
- **CountVectorsFeaturizer** (word and character): Two instances are used in parallel. The first extracts word n-grams up to trigrams, capturing recurring phrases; the second operates on character substrings, adding robustness to morphological variation and minor misspellings.
- **RegexFeaturizer**: Computes additional hint features from manually defined regular expression patterns, providing a lightweight way to inject domain-specific cues.

- **DIET Classifier**: The intent classification head, trained for 250 epochs with a two-layer transformer of dimension 128. Dropout and weight sparsity are applied to regularise the model.
- **Fallback Classifier**: Intercepts any prediction whose confidence falls below a threshold of 0.75, mapping it to a generic fallback intent rather than forcing a low-confidence classification.

6.1.2 Intents and Slots

For this chatbot, five main intents were selected. The intended context of the situation is the player getting caught by a guard and formulating an excuse. As such, the following intents were chosen:

- **Flattery**, example: *"You're clearly the best guard here"*
- **Aggression**, example: *"Don't you dare touch me"*
- **Logic**, example: *"My badge is valid for this sector"*
- **Pity**, example: *"Please have mercy on me"*
- **Bribe**, example: *"I can pay you off to look the other way"*

In order to detect them, a set of around 70-100 example phrases were provided for each intent. The phrases were mostly written by hand, with the help of the autocomplete feature of the editor, and as such many of them are variations of the same phrase.

A memory slot is used to store the name of the current agent that the player is talking to. This will be used to determine the answer of the bot.

6.1.3 Answer Selection and Score Computation

Table 1: Intent weights and sentiment direction for each guard personality profile. Weights range from -1.0 (strongly penalised) to $+1.0$ (strongly rewarded).

Guard	Flattery	Bribe	Pity	Logic	Aggression	Sentiment
Rosanna	$+1.0$	$+0.5$	-0.5	$+0.2$	-1.0	positive
Susanna	$+0.2$	-0.5	$+0.8$	$+0.9$	-1.0	positive
Polyanna	-0.8	-0.5	$+0.7$	-0.2	$+0.4$	negative
Marianna	$+0.6$	$+0.2$	-0.8	-0.6	-0.5	positive

Once the intent has been classified, the answer selection and score computation are handled by a single custom action, `ActionEvaluateExcuse`, defined in `actions.py`. The action is invoked after each player turn and produces both the guard's response line and the sympathy score to be returned to Godot.

The computation proceeds in three steps. First, the action retrieves the current guard's name from the memory slot and uses it to look up a personality profile in `data_skirmish.py`. Each profile contains two fields: a dictionary of `intent_weights`, assigning a value in $[-1, 1]$ to each of the five intents, and a `sentiment_direction` flag, set to either $+1$ or -1. The intent weights encode which persuasion tactics the guard responds to positively; for example, the "Polyanna" instance rewards pity while penalising flattery. The sentiment direction instead encodes the tone of sentences preferred by the agent; for example, "Polyanna" prefers negative tones

Second, a sentiment score is extracted from the raw player text using an external Python library, the VADER analyser, which returns a compound value in $[-1, 1]$ reflecting the overall tone of the message. The final score is then computed as a weighted blend of the two values:

$$s = \underbrace{w_i \cdot \delta_i}_{\text{intent}} + \underbrace{w_s \cdot (\tau \cdot d)}_{\text{sentiment}} \tag{1}$$

where δ_i is the intent weight for the classified intent, τ is the VADER compound tone, d is the guard's sentiment direction, and $w_i = 0.7$, $w_s = 0.3$ are fixed factors giving intent the dominant contribution. The result is clamped to $[-1, 1]$. A score above zero is considered a success.

Third, the score determines which of two pre-written response lines is selected. Each guard has a dedicated response table in `data_skirmish.py`, indexed first by intent and then by the boolean success flag. If the classified intent has no entry for the current guard, the action falls back to the responses for the `logic` intent, which serves as the default. A separate branch handles the `nlu_fallback` case directly, returning a confusion message without consulting the profile at all.

The action's output is a JSON payload sent to Godot, containing the response text, the final sympathy score, and the detected intent.

6.2 Date Chatbot

The Date chatbot is the more complex of the two. While it shares its intent-detection pipeline, it considers two additional factors in its answers. These are two scores kept between interactions with the player: the Trust score and the Suspicion score. The former represents its affinity with the player, the latter its opposite. The objective of the player during its interaction with the bot is to extract a secret, asking "the name of the agent" to the bot. The bot will reveal it only if its Trust is high enough, and suspicion low enough. In order to get to this, the player must first say other things, and try to alter the scores in its favour.

6.2.1 Intents and Slots

For this chatbot, five main intents were selected. The intended context of the situation is the player being on a date with the agent. As such, the following intents were chosen:

- **Compliment**, example: *"I really like your dress"*
- **Curiosity**, example: *"What are your hobbies?"*
- **Talking About Oneself**, example: *"I'm thinking about changing careers"*
- **Toxicity**, example: *"I really can't stand this conversation"*
- **Asking the Objective**, example: *"Tell me the name of the target"*

Similarly to the previous bot, the sentences were mostly hand-written, with the help of autocomplete.

Two memory slots are used to keep track of the Trust and Suspicion scores.

6.2.2 Answer Selection and Score Updates

The answer selection and score update logic for the Date chatbot are handled by the `ActionEvaluateDate` custom action, defined in its own `actions.py`. The structure mirrors the Skirmish evaluator in its general shape, but is more complex, as the scores must persist across turns and the same tactic can produce different results depending on the state of the conversation.

At the start of each turn, the action reads the current Trust and Suspicion values from their respective slots. The new deltas are then computed in two successive layers.

Phase effects. The first layer is a base effect determined by the classified intent and the current *relationship phase*. Trust is partitioned into three named phases: **cold** (below 30), **warm** (below 50), and **close** (above 50). Each intent carries a distinct $(d_{\text{trust}}, d_{\text{sus}})$ pair for each phase, summarised in Table 2. The scores are deliberately reduced toward the lower end of the Trust range, meaning the conversation moves quickly through the cold phase but must work hard to reach and maintain the close phase. The same tactic can have a very different impact depending on how far the conversation has progressed: asking the objective directly carries a trust penalty of -15 and a suspicion spike of $+20$ when the relationship is cold, but yields a small trust gain of $+2$ and only $+7$ suspicion once it is close, reflecting the bot's growing willingness to engage with direct questions. Compliments are inherently high-risk across all phases, always raising suspicion significantly even when they also raise trust. Sharing about oneself, by contrast, is the only intent that reliably reduces suspicion, making it the safest tool for managing the suspicion meter over time.

Tone modulation. For the three social intents (compliment, curiosity, and talking about oneself) the VADER compound score of the player's message is added to the trust delta, scaled by a fixed factor of 2.0. These are the intents where the *delivery* of the message is expected to matter alongside its content. The remaining two intents are immune to tone: asking for a secret or being toxic are evaluated on their face value alone.

Once the deltas have been accumulated across both layers, the scores are updated and clamped to $[0, 100]$. The action then resolves the outcome. There are two terminal conditions: if Suspicion reaches 75, the date ends in failure regardless of any other state; if the player's intent is `ask_objective` and Trust is at or above 80 with Suspicion below 30, the bot reveals the secret and the section concludes with a success. Outside of these two gates, the action selects a response from a fixed table indexed by intent and a tone bucket (`good`, `neutral`, or `bad`) derived from the magnitude of the trust delta: a delta of $+10$ or above maps to `good`, -10 or below to `bad`, and everything in between to `neutral`.

The JSON payload returned to Godot carries the updated Trust and Suspicion values, their deltas, the detected intent, and, in the case of a terminal event, a `game_event` field set to either `date_success` or `date_failed`. On success, the payload also includes the `secret_info` field containing the answer the player was looking for. The updated scores are additionally written back to the Rasa slots via `SlotSet`, so that they are correctly carried into the following turn.

Table 2: Base trust (ΔT) and suspicion (ΔS) deltas per intent and relationship phase. Tone modulation is applied additively on top of these values for compliment, curiosity, and talking about oneself only.

	Cold (< 30)		**Warm** (< 50)		**Close** (≥ 50)	
Intent	ΔT	ΔS	ΔT	ΔS	ΔT	ΔS
Compliment	$+10$	$+13$	$+6$	$+11$	$+2$	$+9$
Curiosity	$+2$	$+3$	$+6$	$+3$	$+10$	0
Talking about self	$+2$	-3	$+6$	-6	$+10$	-3
Toxicity	-9	-2	-13	-5	-20	-9
Ask objective	-15	$+20$	-9	$+13$	$+2$	$+7$

7 Integration

In this section we explain how the different components of the system are integrated with each other, how they're initialized, and how they communicate

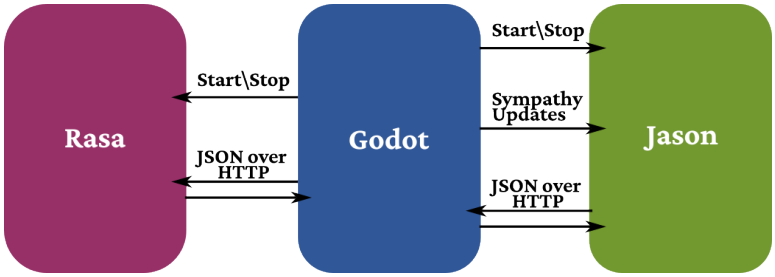


Figure 8: Simple schema summarizing communications between the three main system components

7.1 Initialization

The startup sequence is managed entirely by the Godot side. It proceeds in three ordered stages.

The program begins in the `intro` scene. After the initial menu, the game transitions to a `LoadingScreen` scene, which is responsible for the first stage: starting both Rasa servers. The two chatbots, the Skirmish one on port 8005 and the Date one on port 8006, each with their respective action servers on ports 8055 and 8056, are launched as background processes by Godot, using the local Python environment present in `tongue/venv`. The loading screen then polls each server at regular intervals, sending a lightweight HTTP request to its status endpoint, and only proceeds once both have returned a successful response. The two Rasa servers are kept alive for the entire duration of the game session, to reduce startup overhead.

The second stage begins when the game transitions to the `maze2` scene, where the stealth section is set. At this point, the Jason server is started as a separate process. Unlike the Rasa servers, the Jason server is tightly coupled to the agents' bodies present in the scene: each body registers itself with the server on a unique port at scene initialisation, and the server maintains a live connection to each of them. This coupling means that when the scene is unloaded, either because the player was caught and enters a dialogue section, or because they reach the target, the connections are lost and the server's state becomes inconsistent. The decision was therefore taken to shut down and restart the Jason server on every transition into the stealth scene, rather than attempt to maintain and resynchronise the connections. While this introduces a loading overhead at each restart, it significantly simplifies the connection management logic and eliminates an entire class of potential synchronisation issues.

The third stage of startup is the handshake between Jason and Godot. After the server is started, the `ready_agent` waits one second for the runtime to stabilise, then fires a `vesna.signal_ready` action, which sends a fixed HTTP message to a known Godot endpoint. The game remains on the loading screen until this message is received, at which point it sends back the sympathy update payload which the ready agent distributes to the rest of the squad before any patrolling begins. This step is what allows the personality changes accumulated through chatbot interactions in previous runs to survive the server restart.

Whenever the game session ends, either by completing the game or through an external close event, Godot terminates both the Jason and Rasa processes before exiting.

7.2 Communication

All three external servers communicate with Godot over HTTP, exchanging JSON payloads.

7.2.1 Jason Communication

On the Godot side, each agent body is equipped with a `VesnaManager` node, which initialises the connection to the corresponding Jason mind on startup and exposes a set of helper methods

for sending structured messages. Each agent communicates with its mind on a unique port, agreed upon statically between the two sides at configuration time.

Messages from Godot to Jason are sent as JSON objects, and are translated by the previously mentioned modified `VesnaAgent` into AgentSpeak beliefs added to the receiving agent's memory. In the opposite direction, the internal actions defined in the `vesna/via` folder (`transition_to`, `set_var`, `add_temper`, and `signal_ready`) each structure their arguments into a JSON message and post it to the `VesnaManager` endpoint on the Godot side, which then applies the requested state change or variable update to the body directly. The structure of these messages follows a fixed schema, with a `type` field identifying the action and an `args` field carrying its parameters.

7.2.2 Rasa Communication

The connection to the Rasa servers is managed by a `RasaRequest` node, present in the dialogue scene. When the player submits a message, the node constructs an HTTP request to the `/webhooks/rest/webhook` endpoint of the appropriate server, carrying the player's text and the current sender identifier (which encodes the guard's name for the Skirmish bot, and a fixed session identifier for the Date bot). The response is a JSON array containing the bot's reply text and a `custom` field carrying the structured payload produced by the custom action.

For the Skirmish chatbot, this payload contains three fields:

- `sympathy_score`: the computed persuasion score in $[-1, 1]$, used to update the corresponding agent's mood in Jason on the next stealth section startup.
- `detected_intent`: the classified intent, logged for debugging.
- `guard_name`: echoed back to confirm which personality profile was used.

For the Date chatbot, the payload is larger:

- `ease_score` and `suspicion_score`: the updated meter values, used by Godot to refresh the HUD display.
- `delta_suspicion`: the change in suspicion this turn, used for visual feedback.
- `detected_intent`: the classified intent.
- `game_event`: present only on terminal turns, set to either `date_success` or `date_failed`.
- `secret_info`: present only on success, carrying the answer the player was seeking.

8 Testing

8.1 Software Requirements and Startup

The project's code can be found in its related Github repository [4]. In order to run it, the following software is required:

- Godot Engine Version 4.6+
- Java 23.0
- Gradle 8.14.3
- Jason 3.3.0
- Python 3.10.3 with the following packages: `rasa`, `vaderSentiment`, `spacy`
- Spacy's `en_core_web_lg` model

For the python packages, a local environment has already been created in the `tongue/venv` folder.

As previously mentioned, starting from the `intro` scene, the Godot program should handle the initialization of the programs. The other components are started using the following commands:

- The Jason server can be started by executing `gradle run` in the `mind` folder
- For each Rasa chatbot, both the main and actions server must be run, each on its respective port. For the Skrimish chatbot, one must run in the `tongue/skirmish` folder:
  - `rasa run --enable-api --cors "*" --port 8005`
  - `rasa run actions --port 8055`

  Similarly, for the Date chatbot, one must run, in the `tongue/date` folder:
  - `rasa run --enable-api --cors "*" --port 8006`
  - `rasa run actions --port 8056`

8.2 Results and Issues

The program was tested on mainly two systems, one running Arch Linux, the other Windows 11. The program mostly behaved as expected, with the following issues detected:

- When running the program on the Windows-based system, the Jason and Rasa servers often failed to be correctly initialized by the Godot program. This is thought to be related to the different way in which the Godot launches programs on the Windows version.
- When navigating the maze, agents get sometimes temporarily stuck between each other. This is a problem strictly related to Godot's navigation system. While the problem has been reduced, using Godot's own `NavigationObstacle` nodes, in order to make the agents avoid each other, the problem still subsists.
- Chatbot's results were also tested through crossvalidation (`rasa test nlu --cross-validation --folds 5`). The complete results can be found in the `results` folder in each respective directory. While both chatbots have a relatively high precision, with a macro F1 score of around 0.9, they often fail to generalize, and when they assign incorrect intents, they often do with an high confidence, as visible in the prediction histograms of figures 9 and 10. This is surely related to the quality and most importantly quantity of the example data. While it was searched for, no relevant datasets were found for the intents considered, and even if tested, the usage of LLM-generated sentences was not found to improve the quality of the results. In fact, these sentences often were too varied in their nature, and as such created less distinction between intents. So, their inputs were not used in the final project.
- The Messenger system of the patrols has been difficult to test, given the situation in which it applies being found difficult to replicate. As such, it is not believed to have been tested thoroughly, and may be still hiding some defects.

9 Final Remarks

Ultimately, the project reached its objective, producing a dynamic game system in which recognizing the individual agents rewards the player. Its rules can be easily modified through the minds' personality settings and the chatbots' scoring weights, without needing to touch lower-level systems. The heterogeneous components communicate through standard JSON

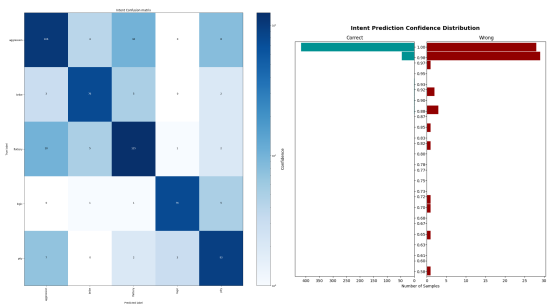


Figure 9: Crossvalidation results for the Skirmish chatbot. intent confusion matrix on the left, and prediction confidence histogram on the right.

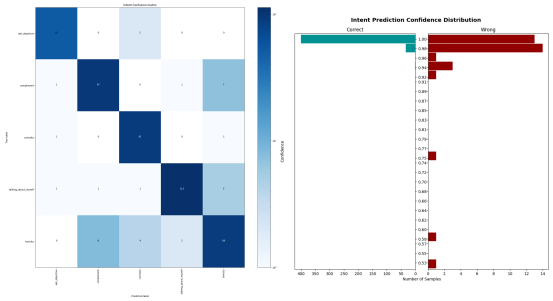


Figure 10: Crossvalidation results for the Date chatbot. intent confusion matrix on the left, and prediction confidence histogram on the right.

payloads and can be upgraded or replaced independently. Still, the project possesses numerous limitations, leaving room for future improvements.

The most significant architectural limitation is the restart overhead introduced by closing and reopening the Jason server at each transition into the stealth scene. A more robust solution would maintain a persistent connection between the mind and the body, passing the updated sympathy values through it rather than reconstructing the full server state at each run. This would also open the possibility of carrying over the agents' fear mood across runs, which is currently reset on every restart and represents a missed opportunity for a more persistent sense of escalation.

On the agent design side, the coordination between agents remains relatively simple. The messenger system is the most sophisticated inter-agent interaction in the current implementation, but the agents otherwise act mostly independently once alerted. The usage of more sophisticated coordination strategies would showcase the advantage of exploiting AgentSpeak for such a task.

The chatbot components, while functional, are limited by the quantity and diversity of their training data. A larger and more varied dataset, possibly supplemented by targeted data augmentation, would likely improve generalisation without sacrificing the intent boundaries. Moreover, the Skirmish chatbot currently treats each interaction as a fully independent episode. Introducing a conversational memory, allowing the agent to reference previous interactions, would deepen the perceived character of the agent.

These improvements, together with a a more polished visual design, would improve the consistency of the game, and the immersion of the player, bringing the current system closer to its original vision.

References

[1] Bandai Namco Entertainment America, "File:Pac-Man Cutscene.svg," *Wikimedia Commons*. [Online]. Available: https://commons.wikimedia.org/w/index.php?curid=165506548. (CC BY 3.0).

[2] Wikipedia contributors, "File:Metal Gear MSX Screen.png," *Wikipedia, The Free Encyclopedia*. [Online]. Available: https://en.wikipedia.org/w/index.php?curid=7390179. (Fair use).

[3] GameFAQs, "Façade Images," *GameSpot*. [Online]. Available: https://gamefaqs.gamespot.com/pc/919887-facade/images?pid=919887&img=145.

[4] Inkeaton. *TakeThemOut*. GitHub Repository.
Available at: https://github.com/inkeaton/TakeThemOut